

01

0.5시간

Claude Code 소개

Agent

Agentic loop

내장 도구

도구 비교 기준

CLAUDE.md

자동 메모리

이 챕터에서 다루는 내용

01 Agent와 Agentic 코딩 도구

자동완성이나 챗봇과 무엇이 다른지 정의부터 정리합니다

02 Cursor, GitHub Copilot과의 차이

우열을 가리지 않고 작업 단위를 기준으로 비교합니다

03 **CLAUDE.md** 와 자동 메모리

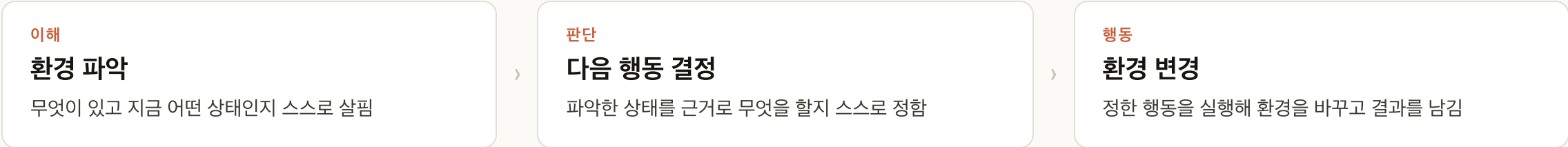
세션마다 빈 상태로 시작하는 컨텍스트를 무엇으로 채우는지 확인합니다

이 내용은 이후 챕터에서 실습으로 다룹니다.

설치와 기본 사용(챕터 2, 3)으로 손에 익히고, **CLAUDE.md** 와 룰(챕터 4)로 맥락을 고정한 뒤, Skill과 Subagent(챕터 5)로 반복 작업을 묶습니다.

Agent의 정의

Agent는 주어진 목표를 이루기 위해 **자신이 속한 환경을 스스로 파악하고, 그 결과를 근거로 판단하며, 환경을 직접 바꾸는** 시스템입니다.



행동으로 환경이 바뀌면 다시 이해 단계로 돌아갑니다. 목표를 이룰 때까지 이 순환이 이어집니다.

빠진 요소

챗봇	환경을 직접 바꾸지 못합니다. 이해하고 판단한 결과를 글로 답할 뿐입니다.
워크플로	판단이 없습니다. 사람이 미리 정해 둔 경로를 그대로 따라 실행합니다.

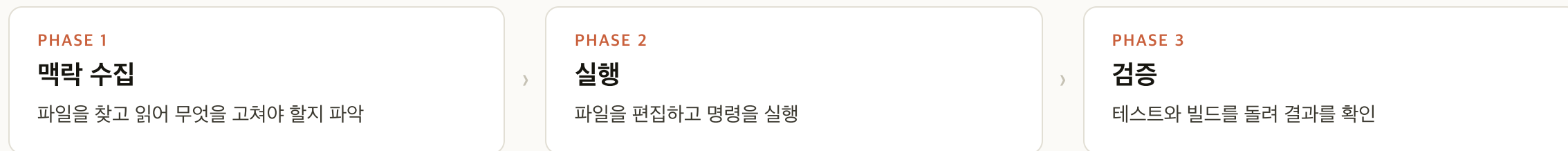
Agentic 코딩 도구, Claude Code

구성 요소	코딩 작업의 예
목표	로그인이 실패하는 버그를 고쳐 달라는 한 문장
환경	코드베이스, 터미널, git 저장소
도구	파일 읽기와 편집, 명령 실행, 코드 검색
검증	테스트와 빌드로 확인하는 통과 여부

- Claude Code는 이 구조를 터미널에 구현한 도구입니다. 코드베이스를 읽고, 파일을 편집하고, 명령을 실행하며, 개발 도구와 연동합니다.
- 모델이 아니라 실행 틀(하네스)입니다. 언어 모델에 도구, 컨텍스트 관리, 실행 환경을 엮어 코딩 Agent로 동작하게 합니다.

터미널, IDE 확장, 데스크톱 앱, 웹이 같은 엔진을 씁니다.
 걸모습이 달라도 `CLAUDE.md` 와 설정, MCP 서버는 그대로 적용됩니다.

Agentic loop



작업이 끝날 때까지 반복하며, 앞 단계에서 알아낸 내용을 근거로 다음 단계를 정합니다.

지시	스스로 밟는 단계
실패하는 테스트를 고쳐 주세요	테스트 실행 → 오류 출력 확인 → 관련 소스 검색 → 파일 읽기 → 수정 → 테스트 재실행

진행 중에 언제든지 개입할 수 있습니다.

`Esc` 를 누르면 그 자리에서 멈춥니다. 누르지 않고 다음 지시를 보내면 하던 동작을 끝낸 뒤 그 내용을 다음 판단에 반영합니다.

다섯 종류의 내장 도구

종류	할 수 있는 일
파일 조작	파일 읽기, 코드 편집, 새 파일 생성
검색	패턴으로 파일 찾기, 정규식으로 내용 검색, 코드베이스 탐색
실행	셸 명령 실행, 서버 기동, 테스트 실행, git 사용, 파일 이동과 이름 변경
웹	웹 검색, 문서 조회, 오류 메시지 검색
작업 관리	할 일 목록 작성과 갱신, Subagent에 작업 위임, Skill 호출

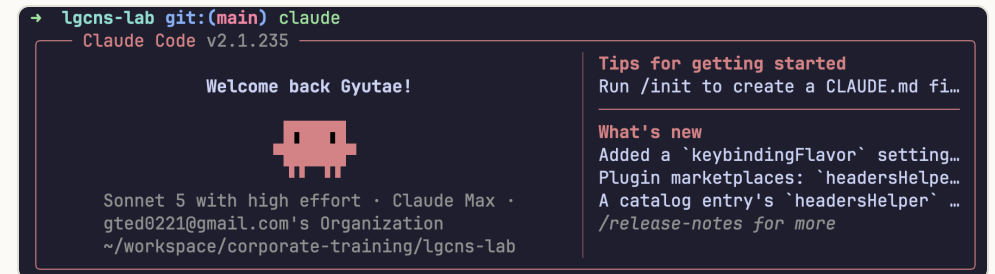
도구가 있어야 Agent가 됩니다.

도구가 없는 언어 모델은 대답만 할 수 있습니다. 반복 작업을 묶는 Skill과 맥락을 나누는 Subagent도 이 도구 목록 안에 있고, MCP는 여기에 외부 도구를 더합니다.

Claude Code 접근 범위

프로젝트	실행한 디렉터리와 하위 파일. 그 밖의 경로는 접근할 때마다 승인 필요
터미널	사용자가 실행할 수 있는 명령 전부. 빌드 도구, git, 패키지 매니저
git 상태	현재 브랜치, 커밋하지 않은 변경, 최근 커밋 이력
CLAUDE.md	세션마다 자동으로 읽어 들이는 프로젝트 지침
자동 메모리	이전 세션에서 스스로 적어 둔 학습 내용

프로젝트 전체가 보이기 때문에 **여러 파일에 걸친 작업을** 한 번에 처리합니다. 다만 볼 수 있다고 해서 전부 읽어 들이지는 않습니다. 필요한 파일만 찾아 읽습니다.



실행 화면 하단에 모델명과 작업 디렉터리 표시

권한 모드와 되돌리기

권한 모드	동작
manual	파일 편집과 명령 실행 전에 매번 확인
accept edits	파일 편집은 자동 승인, 그 밖의 명령은 확인
plan	탐색과 계획만 하고 소스는 고치지 않음
auto	백그라운드 안전 검사를 거쳐 스스로 진행

Shift+Tab 으로 모드를 바꿉니다.

되돌리기는 모드와 무관합니다. 파일을 고치기 직전에 원래 내용이 저장되므로, **Esc** 를 두 번 누르면 이전 상태로 돌아갑니다.

```

프로젝트 내 모든 파일들의 내용을 파악하고, 전체 구성에 대해 요약하여 root 디렉토리에 summary.md 파일로 출력해.

Searched for 1 pattern (ctrl+o to expand)

● Project is a moderate-sized Python codebase. Let me read the key files to build the summary.

Read 5 files (ctrl+o to expand)

● Now let me read the source code files (the skeleton) and test files.

Read 13 files (ctrl+o to expand)

● Bash(echo "--- .gitignore ---"; cat .gitignore; echo; echo "--- .gitattributes ---"; cat .gitattributes; echo; echo "---
sample head ---"; head -3 samples/연구소.csv; echo; echo "--- fixture r001 ---"; cat tests/rules/fixtures/r001.csv;
echo; echo "--- __init__ files ---"; for f in $(find src -name "__init__.py"); do echo "=== $f ==="; cat "$f"; done
Inspect gitignore, gitattributes, sample data, fixture, and init files

Bash command

echo "--- .gitignore ---"; cat .gitignore; echo; echo "--- .gitattributes ---"; cat .gitattributes; echo; echo "---
sample head ---"; head -3 samples/연구소.csv; echo; echo "--- fixture r001 ---"; cat tests/rules/fixtures/r001.csv;
echo; echo "--- __init__ files ---"; for f in $(find src -name "__init__.py"); do echo "=== $f ==="; cat "$f"; done
Inspect gitignore, gitattributes, sample data, fixture, and init files

Contains simple_expansion

Do you want to proceed?
) 1. Yes
  2. No

Esc to cancel · Tab to amend · ctrl+e to explain

```

manual 모드에서 명령을 실행하기 전 승인 요청

되돌리기는 Claude Code가 편집 도구로 바꾼 파일만 대상입니다. 셸 명령으로 생긴 변경은 포함되지 않습니다.

데스크톱 앱의 특징

변경 검토	diff를 화면에서 보며 줄 단위로 의견 작성. 이어서 PR의 CI 상태까지 확인
병렬 세션	세션마다 git worktree로 프로젝트 사본이 격리되어 서로 간섭하지 않음
화면 배치	채팅, diff, 브라우저, 터미널, 파일 편집기를 나란히 두고 재배치
보조 질문	세션 맥락을 쓰되 원래 대화에는 남기지 않음
설정 공유	CLI와 같은 엔진이라 <code>CLAUDE.md</code> 와 설정을 그대로 사용
교육 환경	유료 구독이 필요해 교육용 API 키로는 사용 불가. 실습은 CLI로 진행



변경 검토

고정됨

- autopr

Famtography

- Famtography Admin Client
- Famtography Client
- Famtography Infra
- Famtography Media Processor
- Famtography Server
- Famtography Server 2

hexwright

- Hexwright

병렬 세션

Cursor · GitHub Copilot과의 비교

	Claude Code	Cursor	GitHub Copilot
위임 단위	작업 하나를 통째로 위임 탐색 · 수정 · 테스트 · 커밋까지	코드를 쓰는 순간을 보조 Tab 완성과 인라인 편집	기존 편집기에 추가하는 코드 완성 VS Code · JetBrains · Xcode
강점	편집기와 플랫폼에 묶이지 않음 확장 장치가 가장 많음: Hook · Skill · Subagent	편집 중 곧바로 반응 변경 내용이 diff로 즉시 보임	편집기를 바꾸지 않고 도입 조직 단위 배포가 쉬움
약점	Anthropic 모델만 사용 편집 기능이 없어 IDE는 따로 사용	편집기를 교체해야 함 터미널과 CI에서는 이점이 줄어들	편집 보조가 중심 코딩 Agent는 한 저장소 · 59분 한도

Claude Code를 고르는 이유는 말할 수 있는 범위가 가장 넓기 때문입니다.

두 도구는 사람이 코드를 쓰는 속도를 올려 주고, Claude Code는 작업 하나를 통째로 맡아 결과를 돌려줍니다. 모델은 Anthropic으로 고정되지만, 위임 범위를 넓히는 장치를 가장 많이 제공합니다. 이 과정에서는 그 장치를 만드는 데 하루를 씁니다.

위임 범위를 넓히는 네 가지 장치

장치	역할	다루는 챕터
CLAUDE.md	세션마다 비워지는 컨텍스트를 파일로 유지	챕터 1 · 4
Hook	도구 호출 전후에 검사를 강제	챕터 4
Skill	되풀이하는 절차를 묶어 호출	챕터 5
Subagent	별도 컨텍스트에서 조사와 검토를 대행	챕터 5

CLAUDE.md 는 권고이고 Hook은 강제입니다.

이 과정에서는 네 가지를 직접 만들며 무엇을 권고로 두고 무엇을 강제할지 정합니다.

CLAUDE.md의 정의

`CLAUDE.md` 는 프로젝트 루트에 두는 마크다운 파일입니다. Claude Code가 세션을 시작할 때마다 이 파일을 읽어 대화 맨 앞에 씁니다.

- **적는 내용:** 빌드와 테스트 명령, 코딩 컨벤션, 디렉터리 규칙, 쓰면 안 되는 라이브러리처럼 코드만 읽어서는 알 수 없는 약속입니다.
- **대화와의 차이:** 대화로 전한 내용은 컨텍스트가 정리될 때 밀려나지만, 파일에 적은 내용은 다음 세션에도 그대로 실립니다.
- **공유 범위:** 저장소에 함께 올리므로 팀 전체가 같은 지침을 공유합니다.

같은 설명을 두 번 했다면 `CLAUDE.md` 에 적을 때입니다.

지침이 파일에 있으면 설명은 한 번으로 끝납니다.

CLAUDE.md의 위치와 범위

범위	위치	공유 대상
조직	C:\Program Files\ClaudeCode\CLAUDE.md	조직 전체. 개인 설정으로 제외 불가
사용자	C:\Users\사용자명\.claude\CLAUDE.md	본인의 모든 프로젝트
프로젝트	.\CLAUDE.md 또는 .\.claude\CLAUDE.md	저장소를 공유하는 팀 전체
로컬	.\CLAUDE.local.md	본인만. .gitignore 에 등록

Claude Code는 네 곳의 파일을 순서대로 읽어 **덮어쓰지 않고 전부 이어 붙입니다.**

실행한 디렉터리에서 상위로 올라가며 찾고, 하위 디렉터리에 있는 **CLAUDE.md** 는 그 디렉터리의 파일을 읽을 때 함께 불러옵니다. 실제로 무엇을 읽어 들었는지는 **/context** 를 실행해 확인합니다.

CLAUDE.md 작성 기준

DO 읽거나 추측해서는 알 수 없는 것

- 프로젝트 고유의 빌드와 테스트 명령
- 기본값과 다른 코드 스타일 규칙
- 선호하는 테스트 러너와 실행 방법
- 브랜치 이름과 PR 규칙
- 이 프로젝트만의 아키텍처 결정
- 필수 환경변수 같은 개발 환경 특이사항

DON'T 읽거나 추측해서 알 수 있는 것

- 코드에 이미 드러나 있는 내용
- 언어의 표준 관례
- 상세한 API 문서 (링크로 대신)
- 자주 바뀌는 정보
- 파일 하나하나에 대한 설명
- 코드를 깔끔하게 작성하라는 수준의 당연한 원칙
- 지켰는지 확인할 수 없는 모호한 지시

200줄 이내를 목표로 작성합니다.

길어지면 정작 중요한 규칙이 묻혀서 지켜지지 않습니다. 상황마다 참고해야 하는 구체적인 지시는 별도 문서에 두고 링크만 남깁니다.

CLAUDE.md와 자동 메모리

	CLAUDE.md	자동 메모리
작성자	사람	Claude
내용	지시와 규칙	학습한 내용과 패턴
저장 위치	프로젝트 루트	<code>.claude/projects\<프로젝트>\>\memory\</code>
세션 로드	파일 전체	<code>MEMORY.md</code> 앞부분만. 주제별 파일은 필요할 때 읽음

둘 다 세션마다 읽어 들이는 파일이지만, 강제 규칙이 아니라 참고 자료입니다. 반드시 지켜야 하는 규칙은 Hook으로 만듭니다. 자동 메모리는 `/memory` 로 열어 보고 고칠 수 있습니다.

```
/context
└─ Context Usage
   ┌─ Sonnet 5
   │  cClaude-sonnet-5
   │  127k/967k tokens (13%)
   └─ Estimated usage by category
      ┌─ System prompt: 9.4k tokens (1.0%)
      │ System tools: 27.8k tokens (2.9%)
      │ Memory files: 36.1k tokens (3.7%)
      │ Skills: 2.7k tokens (0.3%)
      │ Messages: 51k tokens (5.3%)
      │ Free space: 807k (83.4%)
      └─ Autocompact buffer: 33k tokens (3.4%)
```

`/context` 로 확인하는 메모리 파일 목록

```
Memory files · /memory
├─ ~/.claude/CLAUDE.md: 94 tokens
├─ ~/.claude/COMMANDS.md: 2.5k tokens
├─ ~/.claude/FLAGS.md: 3.4k tokens
├─ ~/.claude/PRINCIPLES.md: 3.3k tokens
├─ ~/.claude/RULES.md: 871 tokens
├─ ~/.claude/MCP.md: 4.1k tokens
├─ ~/.claude/PERSONAS.md: 7.5k tokens
├─ ~/.claude/ORCHESTRATOR.md: 9.1k tokens
└─ ~/.claude/MODES.md: 5.3k tokens
```

파일별 점유량

자주 나오는 질문

Agent가 스스로 진행한다면 개발자는 무엇을 하나요?

목표를 세우고 결과를 검증합니다. 무엇을 만들지와 무엇을 통과로 볼지는 사람이 정합니다.

틀린 결과를 내면 어떻게 하나요?

테스트나 빌드처럼 직접 돌려 볼 수 있는 검사를 함께 지정합니다. 검사가 실패하면 스스로 다시 고칩니다.

터미널에 익숙하지 않은데 괜찮나요?

실습은 터미널 CLI로 진행합니다. 명령을 외울 필요는 없고, 필요한 조작은 챕터 3부터 하나씩 함께 다룹니다.

다른 모델도 쓸 수 있나요?

Anthropic 모델 중에서 고릅니다. `/model` 로 바꿀 수 있고, 실습에서는 Sonnet 5를 씁니다.

마음대로 파일을 고치거나 커밋 하나요?

권한 모드가 정합니다. `manual` 모드에서는 편집과 명령마다 승인을 거치며, 편집한 내용은 되돌릴 수 있습니다.

CLAUDE.md 에 적으면 반드시 지키나요?

참고 자료이므로 항상 지킨다고 보장할 수는 없습니다. 반드시 지켜야 하는 규칙은 Hook으로 만듭니다.

질문은 공유해 드린 익명 구글 폼으로 보내 주세요.